

Assignment: Problem Statement · Lab 2

1 Scenario

You are building a small internal tool for the student services team of a university's online course platform. Every day the team receives dozens of messages from students through the helpdesk: complaints about the platform not working, questions or disputes about grades, requests to add/drop or get into a full course, and general questions, all landing in one shared inbox. Right now, a staff member has to read every message, figure out what it's actually about, and write a reply from scratch, which is slow and inconsistent depending on who picks it up.

Your job is to build an AI-powered student helpdesk assistant using LangChain that automatically reads an incoming message, works out what kind of issue it is, and drafts an appropriate first response, the same overall pattern you practiced with the reference files, applied to a new problem.

2 Your Task

Build a Python application using LangChain that:

- Classifies an incoming student message into exactly one of four categories:
 - **technical_issue** : the platform is broken, a page won't load, uploads fail, or something else isn't working
 - **grading_dispute** : the student thinks a grade is wrong, missing, or wants clarification on how it was scored
 - **enrollment_request** : the student wants to add, drop, or get off the waitlist for a course
 - **general_question** : a general how-do-I / clarification question that isn't a technical issue, grade dispute, or enrollment request
- Routes the message to a different prompt/template per category and generates a contextual reply using the LLM.
- Prints the input message, the classification result, and the generated reply clearly to the terminal.

3 Category-Specific Behaviour (required)

Each category's prompt must make the model behave differently, this is the actual point of the assignment, so make each response meaningfully distinct instead of a generic "thanks for reaching out":

Category	What the reply should do
technical_issue	Acknowledge the problem, apologize for the disruption, and ask for details not already given: what device/browser they're using, when it happens, and any error message shown.
grading_dispute	Acknowledge the concern, ask for the assignment/exam name and course if missing, and explain it will be forwarded to the instructor for review.
enrollment_request	Acknowledge the request, ask for the course code/section and student ID if not already given, and explain the next step (approval, waitlist position, etc.).
general_question	Answer helpfully and directly if possible, or point them to where they can find the answer (course handbook, registrar page, etc.).

4 Deliverables

- A single Python file implementing the full pipeline (classification + routing + reply generation).
- Four screenshots, one run per category, each showing input, classification result, and generated reply.